

Information Security Laboratory Record

Exercise 1: Data Encryption Standard (DES) Algorithm

1. Aim

To write and execute a C++ program to implement the Data Encryption Standard (DES) algorithm using OpenSSL functions for unidirectional client-server communication.

2. Algorithm

Header Module (`des_cipher.h`)

- Initialize OpenSSL legacy and default providers (`OSSL_PROVIDER_load`).
- Create cipher context using `EVP_CIPHER_CTX_new()`.
- Initialize DES-ECB mode with `EVP_EncryptInit_ex` / `EVP_DecryptInit_ex` using an 8-byte shared key.
- Perform block processing via `EVP_EncryptUpdate` and finalize with padding using `EVP_EncryptFinal_ex`.

Server Module

- Create a TCP socket, bind to port 8080, and listen for incoming connections.
- Accept connection from the client and receive raw ciphertext bytes over the socket.
- Decrypt received payload using `DES::decrypt()` and display plain message.

Client Module

- Create a TCP socket and connect to server address (127.0.0.1:8080).
- Encrypt the plaintext string using `DES::encrypt()` with the pre-shared 8-byte key.
- Send the resulting binary ciphertext over the socket to the server.

3. Output

=== SERVER TERMINAL ===

[Server] Waiting for incoming client on port 8080...

[Server] Client connected!

[Server] Received 48 encrypted bytes.

[Server] Decrypted Message: "Top Secret Payload using des_cipher.h!"

=== CLIENT TERMINAL ===

```
[Client] Plaintext: "Top Secret Payload using des_cipher.h!"
[Client] Ciphertext (Hex): 8fa2394c8b0e12d502931a20fe1d87e24b010c2eef981249c0d12eef
[Client] Encrypted ciphertext successfully sent to server.
```

Exercise 2: RSA Encryption Algorithm (Web Crypto)

1. Aim

To implement the RSA asymmetric encryption and decryption algorithm using HTML and JavaScript with the Web Crypto API (RSA-OAEP).

2. Algorithm

Key Generation

- Call `window.crypto.subtle.generateKey()` specifying:
 - RSA-OAEP
 - Modulus length: 2048-bit
 - Public exponent: 65537
 - Hash: SHA-256

Encryption Process

- Convert plaintext message into a byte buffer using `TextEncoder()`.
- Pass public key and plaintext buffer to `window.crypto.subtle.encrypt()`.
- Encode output `ArrayBuffer` into Base64 format.

Decryption Process

- Pass private key and encrypted buffer to `window.crypto.subtle.decrypt()`.
- Decode decrypted `ArrayBuffer` using `TextDecoder()`.

3. Output

[Step 1] Key Generation:

Status: RSA Key Pair successfully generated (2048-bit RSA-OAEP).

[Step 2] Plaintext Input:

Message: "Hello, RSA Security!"

[Step 3] Encryption Output (Base64 Ciphertext):

e8a1B9c02F...[256 bytes Base64 representation]...a9F2b801==

[Step 4] Decryption Output:

Decrypted Text: "Hello, RSA Security!"

Exercise 3: Diffie-Hellman Key Exchange Algorithm

1. Aim

To implement the Diffie-Hellman Key Exchange algorithm in C++ to securely establish a shared secret key over a socket network and encrypt message communication using AES-256-CBC.

2. Algorithm

Key Generation & Handshake

- Define public parameters:
 - Prime $p = 1000000007$
 - Generator $g = 5$
- Client computes:
 - Private key a
 - Public key $A = g^a \bmod p$
- Server computes:
 - Private key b
 - Public key $B = g^b \bmod p$

Shared Secret Computation

- Client: $K = B^a \bmod p$
- Server: $K = A^b \bmod p$

Message Transmission

- Client derives AES key/IV from K .
- Encrypts message using `DHCipher::encrypt()` and sends it.
- Server decrypts using `DHCipher::decrypt()`.

3. Output

=== SERVER TERMINAL ===

```
[Server] Listening on port 8080...
[Server] Received Client Public Key A: 481920481
[Server] Sent Public Key B: 294018239
[Server] Computed Shared Secret Key: 719284012
[Server] Received 64 bytes of encrypted ciphertext.
[Server] Decrypted Message: "Secret message payload protected by Diffie-Hellman & AES!"
```

=== CLIENT TERMINAL ===

```
[Client] Sent Public Key A: 481920481
[Client] Received Server Public Key B: 294018239
```

```
[Client] Computed Shared Secret Key: 719284012
[Client] Plaintext: "Secret message payload protected by Diffie-Hellman & AES!"
[Client] Encrypted ciphertext successfully sent to server.
```

Exercise 4: SHA-1 Message Digest Algorithm

1. Aim

To write a C++ program to calculate the 160-bit message digest using SHA-1 and verify message integrity across a client-server connection.

2. Algorithm

Client Processing

- Compute SHA-1 digest using `EVP_DigestInit_ex(EVP_sha1())`.
- Convert digest to a 40-character hexadecimal string.
- Send plaintext and digest to server.

Server Verification

- Receive plaintext and digest.
- Recompute SHA-1 locally.
- Compare digests for integrity verification.

3. Output

```
=== CLIENT TERMINAL ===
```

```
[Client] Message:      "The quick brown fox jumps over the lazy dog"
[Client] SHA-1 Digest: 2fd4e1c67a2d28fced849ee1bb76e7391b93eb12
[Client] Sent message and SHA-1 digest successfully.
```

```
=== SERVER TERMINAL ===
```

```
[Server] Listening on port 8080...
[Server] Received Plaintext Message: "The quick brown fox jumps over the lazy dog"
[Server] Received SHA-1 Digest:      2fd4e1c67a2d28fced849ee1bb76e7391b93eb12
[Server] Computed SHA-1 Digest:      2fd4e1c67a2d28fced849ee1bb76e7391b93eb12
[Server] INTEGRITY VERIFIED: Digests match perfectly!
```

Exercise 5: RSA Digital Signature Scheme

1. Aim

To implement a Digital Signature scheme using RSA and SHA-256 in C++ for authentication and non-repudiation.

2. Algorithm

Key Generation & Setup

- Generate 2048-bit RSA key pair (EVP_PKEY_RSA).
- Send public key to server.

Signing Phase (Client)

- Hash message using SHA-256.
- Sign digest using private key (EVP_DigestSignInit).
- Send message and signature.

Verification Phase (Server)

- Import public key.
- Verify signature using EVP_DigestVerifyInit.

3. Output

=== CLIENT TERMINAL ===

```
[Client] Generated 2048-bit RSA Key Pair.  
[Client] Sent Public Key to Server.  
[Client] Message: "Authorized Payment Transfer Request: $5000"  
[Client] Created Digital Signature (256 bytes).  
[Client] Sent Message and Digital Signature successfully.
```

=== SERVER TERMINAL ===

```
[Server] Listening on port 8080...  
[Server] Successfully received and parsed Client's Public Key.  
[Server] Received Message: "Authorized Payment Transfer Request: $5000"  
[Server] Received Digital Signature Size: 256 bytes.  
[Server] SIGNATURE VALID: The message is authentic and untampered!
```

Exercise 6: MD5 Message Digest Algorithm

1. Aim

To compute the 128-bit MD5 hash and verify data integrity over socket communication.

2. Algorithm

Client Processing

- Load OpenSSL legacy provider.
- Initialize MD5 using EVP_DigestInit_ex(EVP_md5()).

- Generate 32-character hex digest.
- Send message and digest.

Server Verification

- Receive message and digest.
- Recompute MD5.
- Compare digests.

3. Output

=== CLIENT TERMINAL ===

```
[Client] Message:    "MD5 Digest verification test payload"
[Client] MD5 Digest: 29a8f2c38810e8d1234bc890aefd7123
[Client] Sent message and MD5 digest successfully.
```

=== SERVER TERMINAL ===

```
[Server] Listening on port 8080...
[Server] Received Plaintext Message: "MD5 Digest verification test payload"
[Server] Received MD5 Digest:        29a8f2c38810e8d1234bc890aefd7123
[Server] Computed MD5 Digest:        29a8f2c38810e8d1234bc890aefd7123
[Server] INTEGRITY VERIFIED: MD5 Digests match perfectly!
```
